UNIVERSITY PROJECT REPORT

User Activity Monitoring Tool

Submitted By

Raja Ali Shahid

SAP ID: 63158

Submitted To

Sir Yawar

Department of Computer Science

Table of Contents

1. Introduction 3

1.1 What is Process Monitoring? 3

1.2 Why is Monitoring Important? 3

1.3 Project Overview 3

2. Problem Statement 4

2.1 Difficulty in Manual Tracking 4

2.2 No History in Standard Tools 4

2.3 Need for Automated Monitoring 4

3. Objectives 5

4. System Overview 6

5. Working Principle 7

Step 1 — Take Snapshot 7

Step 2 — Compare Snapshots 7
Step 3 — Detect Events 7
Step 4 — Calculate Duration 7
Step 5 — Log and Repeat 7

6. Key Technologies Used 8
6.1 Windows Toolhelp32 API 8
6.2 STL Data Structures 8

7. GUI Design 9
7.1 Interface Components 9
7.2 Design Principles 9

8. Output Explanation 10
8.1 Timestamp Format 10
8.2 Duration Calculation 10

9. Example Output 11

10. Comparison with Task Manager 12

11. Cyber Security Importance 13
11.1 Detection of Suspicious Processes 13
11.2 User Activity Tracking 13
11.3 Incident Response 13

12. Advantages 14

13. Limitations 15

14. Future Improvements 16

15. Conclusion 17
15.1 Learning Outcomes 17

1. Introduction

1.1 What is Process Monitoring?

Process monitoring refers to the practice of observing and recording the programs running on a computer system. It involves tracking which processes are active, when they start, when they end, and how long they run. Process monitoring tools capture a complete picture of system activity over time, unlike standard system utilities which only show a snapshot of the current state.

1.2 Why is Monitoring Important?

Monitoring running processes is essential for several reasons in both individual and enterprise computing environments:

- Detects unauthorized or suspicious programs that may indicate malware or unauthorized access.
- Provides a complete history of system activity for auditing and accountability.

- Supports IT administrators in diagnosing performance issues.
- Enables cybersecurity teams to respond faster to incidents using detailed logs.
- Helps organizations enforce software usage policies by tracking installed and running programs.

1.3 Project Overview

This project presents a Windows-based User Activity Monitoring Tool built using C++ and the Windows API. The tool tracks processes in real time, logs when each process starts and ends, calculates the duration it was running, and saves all activity to a file. It features a simple graphical user interface with Start and Stop buttons to control monitoring sessions.

2. Problem Statement

Despite the availability of built-in system tools such as Windows Task Manager, there are significant limitations in tracking and recording process activity on modern computers. The following problems motivated the development of this project:

2.1 Difficulty in Manual Tracking

Manually observing which programs are running at any given time is impractical for most users. A computer may run dozens or hundreds of processes simultaneously, making manual tracking impossible without automated support.

2.2 No History in Standard Tools

Windows Task Manager and similar tools display only the current state of running processes. They do not record historical activity. Once a process ends, all information about it — including when it started and how long it ran — is permanently lost.

2.3 Need for Automated Monitoring

There is a clear need for a dedicated lightweight tool that can run in the background, automatically detect process events, record timestamps, calculate durations, and save this information to a persistent log file for later review.

> This project directly addresses these gaps by providing a real-time, file-logging process monitor with a simple GUI, designed to run on any standard Windows system.

3. Objectives

The primary objectives of this project are:

- **Track Process Start and End: Detect when new processes appear or disappear from the system in real time.**

The tool compares periodic snapshots of running processes to identify newly started or recently terminated applications.

- **Calculate Duration: Compute how long each monitored process was running.**

By recording both start and end timestamps, the tool calculates and displays the total running duration for each process.

- **Display Logs in GUI: Show activity events in a real-time log window within the application.**

All detected events are immediately displayed in the graphical interface so the user can observe activity live.

- **Save Logs to File: Write all activity records to a persistent log file on disk.**

All events are saved to activity_log.txt, allowing the user to review the full monitoring history even after the application is closed.

4. System Overview

The User Activity Monitoring Tool is a Windows desktop application with the following high-level characteristics:

Component	Description
Platform	Windows OS (Windows 7 and above)
Language	C++ (standard and Win32 API)
Process API	Windows Toolhelp32Snapshot API
Data Structures	C++ STL: std::set, std::map
GUI Framework	Win32 API (native Windows GUI)
File Output	Standard C++ file I/O (fstream)
Log File	activity_log.txt (plain text format)

The tool operates as a standalone executable. No external libraries or framework installations are required. It integrates directly with the Windows operating system to access low-level process information.

5. Working Principle

The tool uses a polling-based approach to detect process changes. The following steps describe the complete monitoring cycle:

Step 1 — Take Snapshot

Using the CreateToolhelp32Snapshot() function from the Windows API, the application captures a complete list of all processes currently running on the system. Each process entry includes its name (executable filename) and a unique process ID (PID).

Step 2 — Compare Snapshots

The current snapshot is compared against the previous snapshot stored in a std::set. Any process present in the new snapshot but absent from the old one is identified as newly started. Any process present in the old snapshot but missing from the new one is identified as terminated.

Step 3 — Detect Events

Based on the comparison:

- If a process is new: a PROCESS STARTED event is logged with the current timestamp.
- If a process has ended: a PROCESS ENDED event is logged with the current timestamp and calculated duration.

Step 4 — Calculate Duration

The start time for each active process is stored in a std::map<string, time_t>. When a process ends, the tool subtracts the stored start time from the end time to calculate how long it ran. The result is displayed in minutes and seconds.

Step 5 — Log and Repeat

All detected events are written to the GUI log display and to activity_log.txt. The snapshot cycle then repeats at a fixed interval (typically every 1-2 seconds) until the user presses Stop.

6. Key Technologies Used

Technology	Purpose	Details
C++	Core programming language	Fast, low-level access to system resources
Windows API (Win32)	System-level process access	CreateToolhelp32Snapshot, Process32First/Next

Technology	Purpose	Details
Toolhelp32Snapshot	Process enumeration	Captures list of all running processes
std::set	Track active processes	Efficiently stores and compares process names
std::map<string, time_t>	Duration tracking	Maps process names to start timestamps
C++ fstream	File logging	Writes all events to activity_log.txt
Win32 GUI (HWND)	User interface	Buttons and ListBox for log display

6.1 Windows Toolhelp32 API

The core mechanism relies on CreateToolhelp32Snapshot(TH32CS_SNAPPROCESS, 0) which returns a handle containing all running process information. The tool then iterates through entries using Process32First() and Process32Next() to collect process names.

6.2 STL Data Structures

C++ Standard Template Library containers are used for efficient process tracking. A std::set<string> maintains the current list of active processes, enabling O(log n) lookup for comparisons. A std::map<string, time_t> stores the start timestamp for each process so duration can be calculated when it ends.

7. GUI Design

The graphical user interface is built using native Win32 API controls, keeping the application lightweight and free of third-party dependencies. The interface is intentionally simple to ensure ease of use.

7.1 Interface Components

- Start Button: Initiates the monitoring session. Triggers the process snapshot cycle.
- Stop Button: Ends the monitoring session and stops the polling timer.
- Log Display Area: A scrollable ListBox control showing real-time process events with timestamps.

7.2 Design Principles

- Minimal and clean layout with no unnecessary elements.
- All important controls visible immediately without scrolling.
- Log area updates automatically as new events are detected.

- No installation or configuration required — runs as a standalone executable.

> The GUI design prioritizes function over form. The goal is to provide clear, readable output that any user can understand without technical knowledge.

8. Output Explanation

The tool produces output in two forms: the real-time GUI log display and the persistent activity_log.txt file. Both contain the same information in the same format.

Output Type	Meaning	Example
PROCESS STARTED	A new process has been detected	[14:01:03] chrome.exe — PROCESS STARTED
PROCESS ENDED	A process has terminated	[14:03:44] notepad.exe — PROCESS ENDED
Duration	How long the process ran	Duration: 1 min 29 sec
Log File	Saved record of all events	activity_log.txt on the local disk

8.1 Timestamp Format

Each event is prefixed with a timestamp in [HH:MM:SS] format using the local system clock. This provides an accurate time reference for each recorded event.

8.2 Duration Calculation

Duration is computed as: End Time - Start Time, using C++ time_t values (seconds since epoch). The result is converted and displayed as minutes and seconds for readability.

9. Example Output

The following is a sample excerpt from an activity_log.txt file produced by the tool during a monitoring session:

```
[2024-11-10 14:01:03] chrome.exe          — PROCESS STARTED
[2024-11-10 14:02:15] notepad.exe         — PROCESS STARTED
[2024-11-10 14:03:44] notepad.exe         — PROCESS ENDED
                       Duration: 1 min 29 sec
[2024-11-10 14:07:22] chrome.exe          — PROCESS ENDED
                       Duration: 6 min 19 sec
[2024-11-10 14:09:05] mspaint.exe         — PROCESS STARTED
[2024-11-10 14:11:00] mspaint.exe         — PROCESS ENDED
                       Duration: 1 min 55 sec
```

In this example, three processes were tracked. Notepad ran for 1 minute and 29 seconds, Chrome ran for 6 minutes and 19 seconds, and MS Paint ran for 1 minute and 55 seconds. Each event line clearly shows the process name and whether it started or ended.

10. Comparison with Task Manager

Windows Task Manager is the standard built-in tool for viewing processes. The following table highlights the key differences between Task Manager and this project:

Feature	Windows Task Manager	This Project
Shows current processes	Yes	Yes
Process history	No	Yes
Start time recording	No	Yes
End time recording	No	Yes
Duration calculation	No	Yes
File logging	No	Yes (activity_log.txt)
CPU / RAM tracking	Yes	No (future work)
Custom to user needs	No	Yes

The key advantage of this project over Task Manager is the persistent history. Task Manager shows only a live view — the moment a process ends, all information about it disappears. This tool fills that gap by maintaining a complete, searchable, and saveable record of all process activity.

11. Cyber Security Importance

Process monitoring is a foundational capability in cybersecurity. This project, although built as a student exercise, demonstrates concepts directly applicable to real-world security tools:

11.1 Detection of Suspicious Processes

Malware and unauthorized software must run as processes. A monitoring tool that logs every process start event makes it possible to identify unexpected or unknown executables that may indicate a security threat.

11.2 User Activity Tracking

In enterprise environments, monitoring which programs employees run supports acceptable use policy enforcement, productivity analysis, and insider threat detection. A log of process activity provides an evidence trail for investigations.

11.3 Incident Response

During a cybersecurity incident, forensic investigators rely on logs to reconstruct what happened. Activity logs produced by tools like this one help determine when malicious software was first executed, how long it ran, and what other processes were active at the same time.

> Real-world SIEM (Security Information and Event Management) systems and EDR (Endpoint Detection and Response) tools use the same fundamental approach: collect process events, timestamp them, and analyze patterns for anomalies.

12. Advantages

- **Simple and Lightweight: No heavy dependencies. Runs as a single executable with minimal CPU and RAM usage.**
- **Real-Time Monitoring: Detects process changes within seconds using a timer-based polling mechanism.**
- **Persistent File Logging: All events are automatically saved to activity_log.txt for future reference.**
- **History Tracking: Unlike Task Manager, maintains a complete record of all process activity during the session.**
- **Duration Calculation: Automatically computes and displays how long each process was running.**
- **Easy to Use: Simple GUI with just two buttons — no technical knowledge required to operate.**
- **No Installation Required: Runs as a standalone Windows executable.**

13. Limitations

- **Windows Only: The project uses Win32 API functions which are exclusive to the Windows operating system. It cannot run on Linux or macOS without significant rewriting.**
- **No CPU or RAM Tracking: The tool only records process start/end events. It does not monitor resource consumption (CPU, memory, disk I/O) for individual processes.**
- **Polling-Based Delay: Process detection relies on a fixed-interval timer. Very short-lived processes that start and end between polling cycles may not be detected.**
- **No Alert System: The tool does not send notifications or trigger alerts when specific processes are detected. All output is passive.**
- **No Filtering: The tool logs all processes indiscriminately, including system processes, which can make logs verbose.**

14. Future Improvements

The following enhancements are planned or proposed for future versions of this tool:

- **CPU and Memory Usage Tracking: Add real-time tracking of CPU percentage and RAM consumption per process, similar to Task Manager.**
- **Alert System: Implement notifications — desktop popups or email alerts — when blacklisted or unknown processes are detected.**
- **Improved UI: Replace the basic Win32 GUI with a modern framework (e.g., Qt or WinForms) for a more polished and user-friendly interface.**
- **Cross-Platform Support: Rewrite core monitoring logic using platform-agnostic libraries to support Linux and macOS.**
- **Database Logging: Replace plain text file logging with a SQLite database, enabling efficient search, filtering, and export of historical records.**
- **Process Whitelist/Blacklist: Allow users to configure lists of allowed and blocked processes, enabling automated response to unauthorized programs.**

15. Conclusion

This project successfully demonstrates the development of a functional User Activity Monitoring Tool for Windows using C++ and the Windows API. The tool addresses a real gap in standard system utilities by providing persistent, timestamped logging of process activity — a capability absent from tools like Task Manager.

The implementation covers all stated objectives: real-time process detection, start/end time recording, duration calculation, GUI display, and file-based logging. The project was built entirely without third-party libraries, relying only on the Windows API and C++ Standard Template Library.

15.1 Learning Outcomes

- Gained practical experience with C++ systems programming and the Windows Win32 API.
- Learned how operating systems expose process information through APIs like Toolhelp32Snapshot.
- Understood the role of process monitoring in cybersecurity and IT administration.
- Practiced GUI development using native Win32 controls.
- Developed skills in file I/O, timestamp handling, and data structure use in C++.
- Learned to design and build a complete, working application from requirements to output.

> This project provides a strong foundation for more advanced security tooling and systems programming work in future courses and professional projects.